

Package:

Package_spec:

```
CREATE OR REPLACE PACKAGE pkg_arya_management AS
```

```
    PROCEDURE insert_arya_usuario (p_id_usuario IN VARCHAR2, p_nome IN VARCHAR2, p_email  
IN VARCHAR2, p_senha IN VARCHAR2, p_data_nasc IN DATE);
```

```
    PROCEDURE update_arya_usuario (p_id_usuario IN VARCHAR2, p_nome IN VARCHAR2, p_email  
IN VARCHAR2, p_senha IN VARCHAR2, p_data_nasc IN DATE);
```

```
    PROCEDURE delete_arya_usuario (p_id_usuario IN VARCHAR2);
```

```
    PROCEDURE insert_arya_endereco (p_id_endereco IN VARCHAR2, p_bairro IN VARCHAR2,  
p_cidade IN VARCHAR2, p_estado IN VARCHAR2, p_pais IN VARCHAR2, p_latitude IN NUMBER,  
p_longitude IN NUMBER);
```

```
    PROCEDURE update_arya_endereco (p_id_endereco IN VARCHAR2, p_bairro IN VARCHAR2,  
p_cidade IN VARCHAR2, p_estado IN VARCHAR2, p_pais IN VARCHAR2, p_latitude IN NUMBER,  
p_longitude IN NUMBER);
```

```
    PROCEDURE delete_arya_endereco (p_id_endereco IN VARCHAR2);
```

```
    PROCEDURE insert_arya_area_operacao (p_id_area_operacao IN VARCHAR2, p_latitude_central  
IN NUMBER, p_longitude_central IN NUMBER);
```

```
    PROCEDURE update_arya_area_operacao (p_id_area_operacao IN VARCHAR2,  
p_latitude_central IN NUMBER, p_longitude_central IN NUMBER);
```

```
    PROCEDURE delete_arya_area_operacao (p_id_area_operacao IN VARCHAR2);
```

```
    PROCEDURE insert_arya_hub_operacional (p_id_hub IN VARCHAR2, p_nome IN VARCHAR2,  
p_status IN VARCHAR2, p_id_endereco IN VARCHAR2);
```

```
    PROCEDURE update_arya_hub_operacional (p_id_hub IN VARCHAR2, p_nome IN VARCHAR2,  
p_status IN VARCHAR2, p_id_endereco IN VARCHAR2);
```

PROCEDURE delete_arya_hub_operacional (p_id_hub IN VARCHAR2);

PROCEDURE insert_arya_drone (p_id_drone IN VARCHAR2, p_id_hub IN VARCHAR2, p_nome IN VARCHAR2, p_status IN VARCHAR2, p_modelo IN VARCHAR2, p_alcanceKM IN NUMBER, p_cargaKg IN NUMBER, p_carregamento IN VARCHAR2);

PROCEDURE update_arya_drone (p_id_drone IN VARCHAR2, p_id_hub IN VARCHAR2, p_nome IN VARCHAR2, p_status IN VARCHAR2, p_modelo IN VARCHAR2, p_alcanceKM IN NUMBER, p_cargaKg IN NUMBER, p_carregamento IN VARCHAR2);

PROCEDURE delete_arya_drone (p_id_drone IN VARCHAR2);

PROCEDURE insert_arya_ocorrencia (p_id_ocorrencia IN VARCHAR2, p_tipo_ocorrencia IN VARCHAR2, p_nivel_severidade IN NUMBER, p_data_ocorrencia IN TIMESTAMP, p_descricao IN CLOB, p_id_usuario IN VARCHAR2, p_id_endereco IN VARCHAR2, p_id_area_operacao IN VARCHAR2);

PROCEDURE update_arya_ocorrencia (p_id_ocorrencia IN VARCHAR2, p_tipo_ocorrencia IN VARCHAR2, p_nivel_severidade IN NUMBER, p_data_ocorrencia IN TIMESTAMP, p_descricao IN CLOB, p_id_usuario IN VARCHAR2, p_id_endereco IN VARCHAR2, p_id_area_operacao IN VARCHAR2);

PROCEDURE delete_arya_ocorrencia (p_id_ocorrencia IN VARCHAR2);

PROCEDURE insert_arya_missao_drone (p_id_missao IN VARCHAR2, p_id_drone IN VARCHAR2, p_id_ocorrencia IN VARCHAR2, p_dataInicio IN TIMESTAMP, p_dataFim IN TIMESTAMP, p_status IN VARCHAR2);

PROCEDURE update_arya_missao_drone (p_id_missao IN VARCHAR2, p_id_drone IN VARCHAR2, p_id_ocorrencia IN VARCHAR2, p_dataInicio IN TIMESTAMP, p_dataFim IN TIMESTAMP, p_status IN VARCHAR2);

PROCEDURE delete_arya_missao_drone (p_id_missao IN VARCHAR2);

FUNCTION fnc_pontuacao_severidade (p_nivel_severidade IN NUMBER) RETURN VARCHAR2;

FUNCTION fnc_ranking_ocorrencias_hub (p_id_hub IN ARYA_HUB_OPERACIONAL.id_hub%TYPE) RETURN NUMBER;

FUNCTION fnc_calcula_risco (p_nivel_severidade IN NUMBER) RETURN NUMBER;

PROCEDURE prc_verificar_e_ativar_drone(p_id_drone IN ARYA_DRONE.id_drone%TYPE);

PROCEDURE prc_avaliar_severidade(p_severidade IN NUMBER);

PROCEDURE prc_contar_drones_status;

PROCEDURE prc_analisar_ocorrencias_usuario(p_id_usuario IN
ARYA_USUARIO.id_usuario%TYPE);

PROCEDURE prc_listar_drones_manutencao;

PROCEDURE prc_listar_ocorrencias_criticas;

PROCEDURE prc_ativar_drones_em_hubs_ativos;

PROCEDURE prc_relatorio_detalhado_ocorrencias;

PROCEDURE prc_listar_hubs_sem_drones;

FUNCTION fnc_rel_contagem_drones_status RETURN SYS_REFCURSOR;

FUNCTION fnc_rel_ocorrencias_tipo_avg_sev RETURN SYS_REFCURSOR;

FUNCTION fnc_rel_drones_por_hub RETURN SYS_REFCURSOR;

FUNCTION fnc_rel_usuarios_rank_ocorrencias RETURN SYS_REFCURSOR;

FUNCTION fnc_rel_ocorrencias_area_avg_sev RETURN SYS_REFCURSOR;

PROCEDURE prc_valida_usuario (p_email IN ARYA_USUARIO.email%TYPE, p_data_nasc IN
ARYA_USUARIO.data_nasc%TYPE);

PROCEDURE prc_valida_endereco (p_latitude IN ARYA_ENDERECO.latitude%TYPE, p_longitude
IN ARYA_ENDERECO.longitude%TYPE);

```
PROCEDURE prc_valida_area_operacao (p_latitude_central IN  
ARYA_AREA_OPERACAO.latitude_central%TYPE, p_longitude_central IN  
ARYA_AREA_OPERACAO.longitude_central%TYPE);
```

```
PROCEDURE prc_valida_hub_operacional (p_status IN  
ARYA_HUB_OPERACIONAL.status%TYPE);
```

```
PROCEDURE prc_valida_drone (p_nome IN ARYA_DRONE.nome%TYPE, p_status IN  
ARYA_DRONE.status%TYPE, p_modelo IN ARYA_DRONE.modelo%TYPE, p_alcanceKM IN  
ARYA_DRONE.alcanceKM%TYPE, p_cargaKg IN ARYA_DRONE.cargaKg%TYPE);
```

```
PROCEDURE prc_valida_missao_drone (p_dataInicio IN  
ARYA_MISSAO_DRONE.dataInicio%TYPE, p_dataFim IN ARYA_MISSAO_DRONE.dataFim%TYPE,  
p_status IN ARYA_MISSAO_DRONE.status%TYPE);
```

```
END pkg_arya_management;
```

Package PKG_ARYA_MANAGEMENT compilado

Package_Body:

```
CREATE OR REPLACE PACKAGE BODY pkg_arya_management AS
```

```
PROCEDURE insert_arya_usuario (p_id_usuario IN VARCHAR2, p_nome IN VARCHAR2, p_email  
IN VARCHAR2, p_senha IN VARCHAR2, p_data_nasc IN DATE) AS
```

```
BEGIN
```

```
INSERT INTO ARYA_USUARIO (id_usuario, nome, email, senha, data_nasc) VALUES  
(p_id_usuario, p_nome, p_email, p_senha, p_data_nasc);
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em insert_arya_usuario: ' ||  
SQLERRM); RAISE;
```

```
END insert_arya_usuario;
```

```
PROCEDURE update_arya_usuario (p_id_usuario IN VARCHAR2, p_nome IN VARCHAR2, p_email  
IN VARCHAR2, p_senha IN VARCHAR2, p_data_nasc IN DATE) AS
```

```
BEGIN
```

```
UPDATE ARYA_USUARIO SET nome = p_nome, email = p_email, senha = p_senha, data_nasc  
= p_data_nasc WHERE id_usuario = p_id_usuario;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em update_arya_usuario: '  
|| SQLERRM); RAISE;
```

```
END update_arya_usuario;
```

```
PROCEDURE delete_arya_usuario (p_id_usuario IN VARCHAR2) AS
```

```
BEGIN
```

```
DELETE FROM ARYA_USUARIO WHERE id_usuario = p_id_usuario;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em delete_arya_usuario: ' ||  
SQLERRM); RAISE;
```

```
END delete_arya_usuario;
```

```
PROCEDURE insert_arya_endereco (p_id_endereco IN VARCHAR2, p_bairro IN VARCHAR2,  
p_cidade IN VARCHAR2, p_estado IN VARCHAR2, p_pais IN VARCHAR2, p_latitude IN NUMBER,  
p_longitude IN NUMBER) AS
```

```
BEGIN
```

```
INSERT INTO ARYA_ENDERECO (id_endereco, bairro, cidade, estado, pais, latitude, longitude)
VALUES (p_id_endereco, p_bairro, p_cidade, p_estado, p_pais, p_latitude, p_longitude);
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em insert_arya_endereco: '
|| SQLERRM); RAISE;
```

```
END insert_arya_endereco;
```

```
PROCEDURE update_arya_endereco (p_id_endereco IN VARCHAR2, p_bairro IN VARCHAR2,
p_cidade IN VARCHAR2, p_estado IN VARCHAR2, p_pais IN VARCHAR2, p_latitude IN NUMBER,
p_longitude IN NUMBER) AS
```

```
BEGIN
```

```
UPDATE ARYA_ENDERECO SET bairro = p_bairro, cidade = p_cidade, estado = p_estado, pais
= p_pais, latitude = p_latitude, longitude = p_longitude WHERE id_endereco = p_id_endereco;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em update_arya_endereco: '
|| SQLERRM); RAISE;
```

```
END update_arya_endereco;
```

```
PROCEDURE delete_arya_endereco (p_id_endereco IN VARCHAR2) AS
```

```
BEGIN
```

```
DELETE FROM ARYA_ENDERECO WHERE id_endereco = p_id_endereco;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em delete_arya_endereco: '
|| SQLERRM); RAISE;
```

```
END delete_arya_endereco;
```

```
PROCEDURE insert_arya_area_operacao (p_id_area_operacao IN VARCHAR2, p_latitude_central
IN NUMBER, p_longitude_central IN NUMBER) AS
```

```
BEGIN
```

```
INSERT INTO ARYA_AREA_OPERACAO (id_area_operacao, latitude_central, longitude_central)
VALUES (p_id_area_operacao, p_latitude_central, p_longitude_central);
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em
insert_arya_area_operacao: ' || SQLERRM); RAISE;
```

```
END insert_arya_area_operacao;
```

```
PROCEDURE update_arya_area_operacao (p_id_area_operacao IN VARCHAR2,  
p_latitude_central IN NUMBER, p_longitude_central IN NUMBER) AS
```

```
BEGIN
```

```
UPDATE ARYA_AREA_OPERACAO SET latitude_central = p_latitude_central, longitude_central  
= p_longitude_central WHERE id_area_operacao = p_id_area_operacao;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
update_arya_area_operacao: ' || SQLERRM); RAISE;
```

```
END update_arya_area_operacao;
```

```
PROCEDURE delete_arya_area_operacao (p_id_area_operacao IN VARCHAR2) AS
```

```
BEGIN
```

```
DELETE FROM ARYA_AREA_OPERACAO WHERE id_area_operacao = p_id_area_operacao;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
delete_arya_area_operacao: ' || SQLERRM); RAISE;
```

```
END delete_arya_area_operacao;
```

```
PROCEDURE insert_arya_hub_operacional (p_id_hub IN VARCHAR2, p_nome IN VARCHAR2,  
p_status IN VARCHAR2, p_id_endereco IN VARCHAR2) IS
```

```
BEGIN
```

```
INSERT INTO ARYA_HUB_OPERACIONAL (id_hub, nome, status, id_endereco) VALUES  
(p_id_hub, p_nome, p_status, p_id_endereco);
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
insert_arya_hub_operacional: ' || SQLERRM); RAISE;
```

```
END insert_arya_hub_operacional;
```

```
PROCEDURE update_arya_hub_operacional (p_id_hub IN VARCHAR2, p_nome IN VARCHAR2,  
p_status IN VARCHAR2, p_id_endereco IN VARCHAR2) IS
```

BEGIN

UPDATE ARYA_HUB_OPERACIONAL SET nome = p_nome, status = p_status, id_endereco = p_id_endereco WHERE id_hub = p_id_hub;

EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em update_arya_hub_operacional: ' || SQLERRM); RAISE;

END update_arya_hub_operacional;

PROCEDURE delete_arya_hub_operacional (p_id_hub IN VARCHAR2) IS

BEGIN

DELETE FROM ARYA_HUB_OPERACIONAL WHERE id_hub = p_id_hub;

EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em delete_arya_hub_operacional: ' || SQLERRM); RAISE;

END delete_arya_hub_operacional;

PROCEDURE insert_arya_drone (p_id_drone IN VARCHAR2, p_id_hub IN VARCHAR2, p_nome IN VARCHAR2, p_status IN VARCHAR2, p_modelo IN VARCHAR2, p_alcanceKM IN NUMBER, p_cargaKg IN NUMBER, p_carregamento IN VARCHAR2) IS

BEGIN

INSERT INTO ARYA_DRONE (id_drone, id_hub, nome, status, modelo, alcanceKM, cargaKg, carregamento) VALUES (p_id_drone, p_id_hub, p_nome, p_status, p_modelo, p_alcanceKM, p_cargaKg, p_carregamento);

EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em insert_arya_drone: ' || SQLERRM); RAISE;

END insert_arya_drone;

PROCEDURE update_arya_drone (p_id_drone IN VARCHAR2, p_id_hub IN VARCHAR2, p_nome IN VARCHAR2, p_status IN VARCHAR2, p_modelo IN VARCHAR2, p_alcanceKM IN NUMBER, p_cargaKg IN NUMBER, p_carregamento IN VARCHAR2) IS

BEGIN


```
UPDATE ARYA_DRONE SET id_hub = p_id_hub, nome = p_nome, status = p_status, modelo =  
p_modelo, alcanceKM = p_alcanceKM, cargaKg = p_cargaKg, carregamento = p_carregamento  
WHERE id_drone = p_id_drone;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em update_arya_drone: ' ||  
SQLERRM); RAISE;
```

```
END update_arya_drone;
```

```
PROCEDURE delete_arya_drone (p_id_drone IN VARCHAR2) IS
```

```
BEGIN
```

```
DELETE FROM ARYA_DRONE WHERE id_drone = p_id_drone;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em delete_arya_drone: ' ||  
SQLERRM); RAISE;
```

```
END delete_arya_drone;
```

```
PROCEDURE insert_arya_ocorrencia (p_id_ocorrencia IN VARCHAR2, p_tipo_ocorrencia IN  
VARCHAR2, p_nivel_severidade IN NUMBER, p_data_ocorrencia IN TIMESTAMP, p_descricao IN  
CLOB, p_id_usuario IN VARCHAR2, p_id_endereco IN VARCHAR2, p_id_area_operacao IN  
VARCHAR2) IS
```

```
BEGIN
```

```
INSERT INTO ARYA_OCORRENCIA (id_ocorrencia, tipo_ocorrencia, nivel_severidade,  
data_ocorrencia, descricao, id_usuario, id_endereco, id_area_operacao) VALUES (p_id_ocorrencia,  
p_tipo_ocorrencia, p_nivel_severidade, p_data_ocorrencia, p_descricao, p_id_usuario,  
p_id_endereco, p_id_area_operacao);
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em insert_arya_ocorrencia: '  
|| SQLERRM); RAISE;
```

```
END insert_arya_ocorrencia;
```

```
PROCEDURE update_arya_ocorrencia (p_id_ocorrencia IN VARCHAR2, p_tipo_ocorrencia IN  
VARCHAR2, p_nivel_severidade IN NUMBER, p_data_ocorrencia IN TIMESTAMP, p_descricao IN  
CLOB, p_id_usuario IN VARCHAR2, p_id_endereco IN VARCHAR2, p_id_area_operacao IN  
VARCHAR2) IS
```

BEGIN

UPDATE ARYA_OCORRENCIA SET tipo_ocorrencia = p_tipo_ocorrencia, nivel_severidade = p_nivel_severidade, data_ocorrencia = p_data_ocorrencia, descricao = p_descricao, id_usuario = p_id_usuario, id_endereco = p_id_endereco, id_area_operacao = p_id_area_operacao WHERE id_ocorrencia = p_id_ocorrencia;

EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em update_arya_ocorrencia: ' || SQLERRM); RAISE;

END update_arya_ocorrencia;

PROCEDURE delete_arya_ocorrencia (p_id_ocorrencia IN VARCHAR2) IS

BEGIN

DELETE FROM ARYA_OCORRENCIA WHERE id_ocorrencia = p_id_ocorrencia;

EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em delete_arya_ocorrencia: ' || SQLERRM); RAISE;

END delete_arya_ocorrencia;

PROCEDURE insert_arya_missao_drone (p_id_missao IN VARCHAR2, p_id_drone IN VARCHAR2, p_id_ocorrencia IN VARCHAR2, p_dataInicio IN TIMESTAMP, p_dataFim IN TIMESTAMP, p_status IN VARCHAR2) IS

BEGIN

INSERT INTO ARYA_MISSAO_DRONE (id_missao, id_drone, id_ocorrencia, dataInicio, dataFim, status) VALUES (p_id_missao, p_id_drone, p_id_ocorrencia, p_dataInicio, p_dataFim, p_status);

EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em insert_arya_missao_drone: ' || SQLERRM); RAISE;

END insert_arya_missao_drone;

PROCEDURE update_arya_missao_drone (p_id_missao IN VARCHAR2, p_id_drone IN VARCHAR2, p_id_ocorrencia IN VARCHAR2, p_dataInicio IN TIMESTAMP, p_dataFim IN TIMESTAMP, p_status IN VARCHAR2) IS

BEGIN

```
UPDATE ARYA_MISSAO_DRONE SET id_drone = p_id_drone, id_ocorrencia = p_id_ocorrencia,  
dataInicio = p_dataInicio, dataFim = p_dataFim, status = p_status WHERE id_missao =  
p_id_missao;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
update_arya_missao_drone: ' || SQLERRM); RAISE;
```

```
END update_arya_missao_drone;
```

```
PROCEDURE delete_arya_missao_drone (p_id_missao IN VARCHAR2) IS
```

```
BEGIN
```

```
DELETE FROM ARYA_MISSAO_DRONE WHERE id_missao = p_id_missao;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
delete_arya_missao_drone: ' || SQLERRM); RAISE;
```

```
END delete_arya_missao_drone;
```

```
FUNCTION fnc_pontuacao_severidade (p_nivel_severidade IN NUMBER) RETURN VARCHAR2 IS
```

```
v_pontuacao VARCHAR2(10);
```

```
BEGIN
```

```
IF p_nivel_severidade <= 3 THEN v_pontuacao := 'Baixo';
```

```
ELSIF p_nivel_severidade <= 7 THEN v_pontuacao := 'Médio';
```

```
ELSE v_pontuacao := 'Alto';
```

```
END IF;
```

```
RETURN v_pontuacao;
```

```
END fnc_pontuacao_severidade;
```

```
FUNCTION fnc_ranking_ocorrencias_hub (p_id_hub IN  
ARYA_HUB_OPERACIONAL.id_hub%TYPE) RETURN NUMBER IS
```

```
v_total NUMBER;
```

```
BEGIN
```

```
SELECT COUNT(o.id_ocorrencia) INTO v_total FROM ARYA_OCORRENCIA o JOIN  
ARYA_HUB_OPERACIONAL h ON o.id_endereco = h.id_endereco WHERE h.id_hub = p_id_hub;  
  
RETURN v_total;  
  
END fnc_ranking_ocorrencias_hub;
```

```
FUNCTION fnc_calcula_risco (p_nivel_severidade IN NUMBER) RETURN NUMBER IS  
  
v_risco NUMBER;  
  
BEGIN  
  
v_risco := p_nivel_severidade * 1.5;  
  
RETURN v_risco;  
  
END fnc_calcula_risco;
```

```
PROCEDURE prc_verificar_e_ativar_drone(p_id_drone IN ARYA_DRONE.id_drone%TYPE) IS  
  
v_status ARYA_DRONE.status%TYPE;  
  
v_hub_status ARYA_HUB_OPERACIONAL.status%TYPE;  
  
BEGIN  
  
SELECT d.status, h.status INTO v_status, v_hub_status FROM ARYA_DRONE d JOIN  
ARYA_HUB_OPERACIONAL h ON d.id_hub = h.id_hub WHERE d.id_drone = p_id_drone;  
  
IF LOWER(v_status) = 'inativo' AND LOWER(v_hub_status) = 'ativo' THEN  
  
UPDATE ARYA_DRONE SET status = 'ativo' WHERE id_drone = p_id_drone;  
  
DBMS_OUTPUT.PUT_LINE('Drone ativado com sucesso!');  
  
ELSE  
  
DBMS_OUTPUT.PUT_LINE('Drone não pode ser ativado. Verifique status do drone ou do  
hub.');
```

```
END IF;
```

```
EXCEPTION WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Erro: Drone com ID ' ||  
p_id_drone || ' não encontrado.');
```

```
WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
prc_verificar_e_ativar_drone: ' || SQLERRM); RAISE;
```

```
END prc_verificar_e_ativar_drone;
```

```
PROCEDURE prc_avaliar_severidade(p_severidade IN NUMBER) IS
```

```
BEGIN
```

```
IF p_severidade BETWEEN 1 AND 3 THEN DBMS_OUTPUT.PUT_LINE('Severidade Leve');
```

```
ELSIF p_severidade BETWEEN 4 AND 6 THEN DBMS_OUTPUT.PUT_LINE('Severidade  
Moderada');
```

```
ELSIF p_severidade BETWEEN 7 AND 10 THEN DBMS_OUTPUT.PUT_LINE('Severidade Grave');
```

```
ELSE DBMS_OUTPUT.PUT_LINE('Valor de severidade inválido: ' || p_severidade);
```

```
END IF;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em prc_avaliar_severidade: '  
|| SQLERRM); RAISE;
```

```
END prc_avaliar_severidade;
```

```
PROCEDURE prc_contar_drones_status IS
```

```
v_total_ativos NUMBER;
```

```
v_total_em_voo NUMBER;
```

```
BEGIN
```

```
SELECT SUM(CASE WHEN LOWER(status) = 'ativo' THEN 1 ELSE 0 END), SUM(CASE WHEN  
LOWER(status) = 'em voo' THEN 1 ELSE 0 END) INTO v_total_ativos, v_total_em_voo FROM  
ARYA_DRONE;
```

```
DBMS_OUTPUT.PUT_LINE('Drones ativos: ' || NVL(v_total_ativos, 0));
```

```
IF NVL(v_total_em_voo, 0) = 0 THEN DBMS_OUTPUT.PUT_LINE('Nenhum drone em voo no  
momento!');
```

```
ELSE DBMS_OUTPUT.PUT_LINE('Drones em voo: ' || v_total_em_voo);
```

```
END IF;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
prc_contar_drones_status: ' || SQLERRM); RAISE;
```

```
END prc_contar_drones_status;
```

```
PROCEDURE prc_analisar_ocorrencias_usuario(p_id_usuario IN  
ARYA_USUARIO.id_usuario%TYPE) IS
```

```
    v_total_criticas NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO v_total_criticas FROM ARYA_OCORRENCIA WHERE id_usuario =  
p_id_usuario AND nivel_severidade > 7;
```

```
    IF v_total_criticas > 3 THEN DBMS_OUTPUT.PUT_LINE('Usuário com muitas ocorrências  
críticas (' || v_total_criticas || ')!');
```

```
    ELSE DBMS_OUTPUT.PUT_LINE('Ocorrências críticas para o usuário sob controle. Total: ' ||  
v_total_criticas);
```

```
END IF;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
prc_analisar_ocorrencias_usuario: ' || SQLERRM); RAISE;
```

```
END prc_analisar_ocorrencias_usuario;
```

```
PROCEDURE prc_listar_drones_manutencao AS
```

```
    CURSOR c_drones_manutencao IS SELECT id_drone, nome, status FROM ARYA_DRONE  
WHERE LOWER(status) = 'manutencao';
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('--- DRONES EM MANUTENÇÃO ---');
```

```
    FOR rec IN c_drones_manutencao LOOP DBMS_OUTPUT.PUT_LINE('ID: ' || rec.id_drone || '  
| Nome: ' || rec.nome || ' | Status: ' || rec.status); END LOOP;
```

```
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
prc_listar_drones_manutencao: ' || SQLERRM); RAISE;
```

```
END prc_listar_drones_manutencao;
```

```
PROCEDURE prc_listar_ocorrencias_criticas AS
```

```
    CURSOR c_ocorrencias_severas IS SELECT id_ocorrencia, tipo_ocorrencia, nivel_severidade  
FROM ARYA_OCORRENCIA WHERE nivel_severidade > 7;
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('--- OCORRÊNCIAS CRÍTICAS (Severidade > 7) ---');
```

```
    FOR rec IN c_ocorrencias_severas LOOP DBMS_OUTPUT.PUT_LINE('ID: ' || rec.id_ocorrencia  
|| ' | Tipo: ' || rec.tipo_ocorrencia || ' | Severidade: ' || rec.nivel_severidade); END LOOP;
```

```
    EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
prc_listar_ocorrencias_criticas: ' || SQLERRM); RAISE;
```

```
END prc_listar_ocorrencias_criticas;
```

```
PROCEDURE prc_ativar_drones_em_hubs_ativos IS
```

```
    v_drones_ativados NUMBER;
```

```
BEGIN
```

```
    UPDATE ARYA_DRONE SET status = 'ativo' WHERE status = 'inativo' AND id_hub IN (SELECT  
id_hub FROM ARYA_HUB_OPERACIONAL WHERE status = 'ativo');
```

```
    v_drones_ativados := SQL%ROWCOUNT;
```

```
    DBMS_OUTPUT.PUT_LINE('Operação concluída. Total de drones ativados: ' ||  
v_drones_ativados);
```

```
    EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em  
prc_ativar_drones_em_hubs_ativos: ' || SQLERRM); RAISE;
```

```
END prc_ativar_drones_em_hubs_ativos;
```

```
PROCEDURE prc_relatorio_detalhado_ocorrencias IS
```

```
    CURSOR c_usuarios IS SELECT u.id_usuario, u.nome, COUNT(o.id_ocorrencia) AS  
total_ocorrencias FROM ARYA_USUARIO u LEFT JOIN ARYA_OCORRENCIA o ON u.id_usuario =  
o.id_usuario GROUP BY u.id_usuario, u.nome;
```

```
BEGIN
```

```

DBMS_OUTPUT.PUT_LINE('--- RELATÓRIO DETALHADO DE OCORRÊNCIAS POR USUÁRIO ---');

FOR r IN c_usuarios LOOP

    IF r.total_ocorrencias = 0 THEN CONTINUE;

    ELSIF r.total_ocorrencias > 5 THEN DBMS_OUTPUT.PUT_LINE('ATENÇÃO: Usuário ' ||
r.nome || ' tem ' || r.total_ocorrencias || ' ocorrências!');

    ELSE DBMS_OUTPUT.PUT_LINE('Usuário: ' || r.nome || ' | Ocorrências: ' ||
r.total_ocorrencias);

    END IF;

END LOOP;

EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em
prc_relatorio_detalhado_ocorrencias: ' || SQLERRM); RAISE;

END prc_relatorio_detalhado_ocorrencias;

```

```

PROCEDURE prc_listar_hubs_sem_drones AS

    CURSOR c_hubs_sem_drones IS SELECT h.id_hub, h.nome FROM ARYA_HUB_OPERACIONAL
h WHERE NOT EXISTS (SELECT 1 FROM ARYA_DRONE d WHERE d.id_hub = h.id_hub);

BEGIN

    DBMS_OUTPUT.PUT_LINE('--- HUBs SEM DRONES ASSOCIADOS ---');

    FOR r IN c_hubs_sem_drones LOOP DBMS_OUTPUT.PUT_LINE('Hub: ' || r.nome || '(' ||
r.id_hub || ')'); END LOOP;

    EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Erro em
prc_listar_hubs_sem_drones: ' || SQLERRM); RAISE;

END prc_listar_hubs_sem_drones;

```

```

FUNCTION fnc_rel_contagem_drones_status RETURN SYS_REFCURSOR AS o_ref_cursor
SYS_REFCURSOR;

BEGIN OPEN o_ref_cursor FOR SELECT status, COUNT(*) AS total_drones FROM ARYA_DRONE
GROUP BY status ORDER BY total_drones DESC; RETURN o_ref_cursor;

EXCEPTION WHEN OTHERS THEN RAISE; END fnc_rel_contagem_drones_status;

```



```
FUNCTION fnc_rel_ocorrencias_tipo_avg_sev RETURN SYS_REFCURSOR AS o_ref_cursor  
SYS_REFCURSOR;
```

```
BEGIN OPEN o_ref_cursor FOR SELECT tipo_ocorrencia, COUNT(*) AS total_ocorrencias,  
AVG(nivel_severidade) AS severidade_media FROM ARYA_OCORRENCIA GROUP BY  
tipo_ocorrencia ORDER BY total_ocorrencias DESC; RETURN o_ref_cursor;
```

```
EXCEPTION WHEN OTHERS THEN RAISE; END fnc_rel_ocorrencias_tipo_avg_sev;
```

```
FUNCTION fnc_rel_drones_por_hub RETURN SYS_REFCURSOR AS o_ref_cursor  
SYS_REFCURSOR;
```

```
BEGIN OPEN o_ref_cursor FOR SELECT h.id_hub, h.nome AS nome_hub, h.status AS  
status_hub, COUNT(d.id_drone) AS total_drones FROM ARYA_HUB_OPERACIONAL h LEFT JOIN  
ARYA_DRONE d ON h.id_hub = d.id_hub GROUP BY h.id_hub, h.nome, h.status ORDER BY  
total_drones DESC; RETURN o_ref_cursor;
```

```
EXCEPTION WHEN OTHERS THEN RAISE; END fnc_rel_drones_por_hub;
```

```
FUNCTION fnc_rel_usuarios_rank_ocorrencias RETURN SYS_REFCURSOR AS o_ref_cursor  
SYS_REFCURSOR;
```

```
BEGIN OPEN o_ref_cursor FOR SELECT u.id_usuario, u.nome, COUNT(o.id_ocorrencia) AS  
total_ocorrencias FROM ARYA_USUARIO u LEFT JOIN ARYA_OCORRENCIA o ON u.id_usuario =  
o.id_usuario GROUP BY u.id_usuario, u.nome ORDER BY total_ocorrencias DESC; RETURN  
o_ref_cursor;
```

```
EXCEPTION WHEN OTHERS THEN RAISE; END fnc_rel_usuarios_rank_ocorrencias;
```

```
FUNCTION fnc_rel_ocorrencias_area_avg_sev RETURN SYS_REFCURSOR AS o_ref_cursor  
SYS_REFCURSOR;
```

```
BEGIN OPEN o_ref_cursor FOR SELECT a.id_area_operacao, COUNT(o.id_ocorrencia) AS  
total_ocorrencias, AVG(o.nivel_severidade) AS severidade_media FROM ARYA_AREA_OPERACAO a  
LEFT JOIN ARYA_OCORRENCIA o ON a.id_area_operacao = o.id_area_operacao GROUP BY  
a.id_area_operacao ORDER BY severidade_media DESC; RETURN o_ref_cursor;
```

```
EXCEPTION WHEN OTHERS THEN RAISE; END fnc_rel_ocorrencias_area_avg_sev;
```

```
PROCEDURE prc_valida_usuario (p_email IN ARYA_USUARIO.email%TYPE, p_data_nasc IN ARYA_USUARIO.data_nasc%TYPE) IS
```

```
    v_email_pattern CONSTANT VARCHAR2(100) := '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$';
```

```
BEGIN
```

```
    IF NOT REGEXP_LIKE(p_email, v_email_pattern) THEN RAISE_APPLICATION_ERROR(-20001, 'Email em formato inválido.');
```

```
    IF p_data_nasc IS NOT NULL AND p_data_nasc > SYSDATE THEN RAISE_APPLICATION_ERROR(-20002, 'Data de nascimento não pode ser no futuro.');
```

```
END prc_valida_usuario;
```

```
PROCEDURE prc_valida_endereco (p_latitude IN ARYA_ENDERECO.latitude%TYPE, p_longitude IN ARYA_ENDERECO.longitude%TYPE) IS
```

```
BEGIN
```

```
    IF p_latitude IS NOT NULL AND (p_latitude < -90 OR p_latitude > 90) THEN RAISE_APPLICATION_ERROR(-20006, 'Latitude inválida. Deve estar entre -90 e 90.');
```

```
    IF p_longitude IS NOT NULL AND (p_longitude < -180 OR p_longitude > 180) THEN RAISE_APPLICATION_ERROR(-20007, 'Longitude inválida. Deve estar entre -180 e 180.');
```

```
END prc_valida_endereco;
```

```
PROCEDURE prc_valida_area_operacao (p_latitude_central IN ARYA_AREA_OPERACAO.latitude_central%TYPE, p_longitude_central IN ARYA_AREA_OPERACAO.longitude_central%TYPE) IS
```

```
BEGIN
```

```
    IF p_latitude_central < -90 OR p_latitude_central > 90 THEN RAISE_APPLICATION_ERROR(-20008, 'Latitude central inválida. Deve estar entre -90 e 90.');
```

```
    IF p_longitude_central < -180 OR p_longitude_central > 180 THEN RAISE_APPLICATION_ERROR(-20009, 'Longitude central inválida. Deve estar entre -180 e 180.');
```

```
END prc_valida_area_operacao;
```

```
PROCEDURE prc_valida_hub_operacional (p_status IN ARYA_HUB_OPERACIONAL.status%TYPE)  
IS
```

```
BEGIN
```

```
    IF p_status IS NOT NULL AND LOWER(p_status) NOT IN ('ativo', 'inativo', 'manutencao') THEN  
        RAISE_APPLICATION_ERROR(-20010, 'Status inválido para Hub. Use: ativo, inativo, manutencao.');
```

```
    END IF;
```

```
END prc_valida_hub_operacional;
```

```
PROCEDURE prc_valida_drone (p_nome IN ARYA_DRONE.nome%TYPE, p_status IN  
ARYA_DRONE.status%TYPE, p_modelo IN ARYA_DRONE.modelo%TYPE, p_alcanceKM IN  
ARYA_DRONE.alcanceKM%TYPE, p_cargaKg IN ARYA_DRONE.cargaKg%TYPE) IS
```

```
BEGIN
```

```
    IF p_nome IS NULL OR TRIM(p_nome) IS NULL THEN RAISE_APPLICATION_ERROR(-20011,  
'Nome do drone é obrigatório.');
```

```
    END IF;
```

```
    IF p_status IS NOT NULL AND LOWER(p_status) NOT IN ('ativo', 'inativo', 'em voo',  
'manutencao') THEN RAISE_APPLICATION_ERROR(-20012, 'Status inválido para Drone. Use: ativo,  
inativo, em voo, manutencao.');
```

```
    END IF;
```

```
    IF p_modelo IS NULL OR TRIM(p_modelo) IS NULL THEN RAISE_APPLICATION_ERROR(-20016,  
'Modelo do drone é obrigatório.');
```

```
    END IF;
```

```
    IF p_alcanceKM IS NULL OR p_alcanceKM <= 0 THEN RAISE_APPLICATION_ERROR(-20017,  
'Alcance (KM) do drone deve ser maior que zero.');
```

```
    END IF;
```

```
    IF p_cargaKg IS NULL OR p_cargaKg < 0 THEN RAISE_APPLICATION_ERROR(-20018, 'Carga  
(Kg) do drone não pode ser negativa.');
```

```
    END IF;
```

```
END prc_valida_drone;
```

```
PROCEDURE prc_valida_missao_drone (p_dataInicio IN  
ARYA_MISSAO_DRONE.dataInicio%TYPE, p_dataFim IN ARYA_MISSAO_DRONE.dataFim%TYPE,  
p_status IN ARYA_MISSAO_DRONE.status%TYPE) IS
```

```
BEGIN
```

```
IF p_dataInicio IS NULL THEN RAISE_APPLICATION_ERROR(-20019, 'Data de início da missão é obrigatória.');
```

```
END IF;  
  
IF p_dataFim IS NOT NULL AND p_dataFim < p_dataInicio THEN  
RAISE_APPLICATION_ERROR(-20020, 'Data de fim não pode ser anterior à data de início.');
```

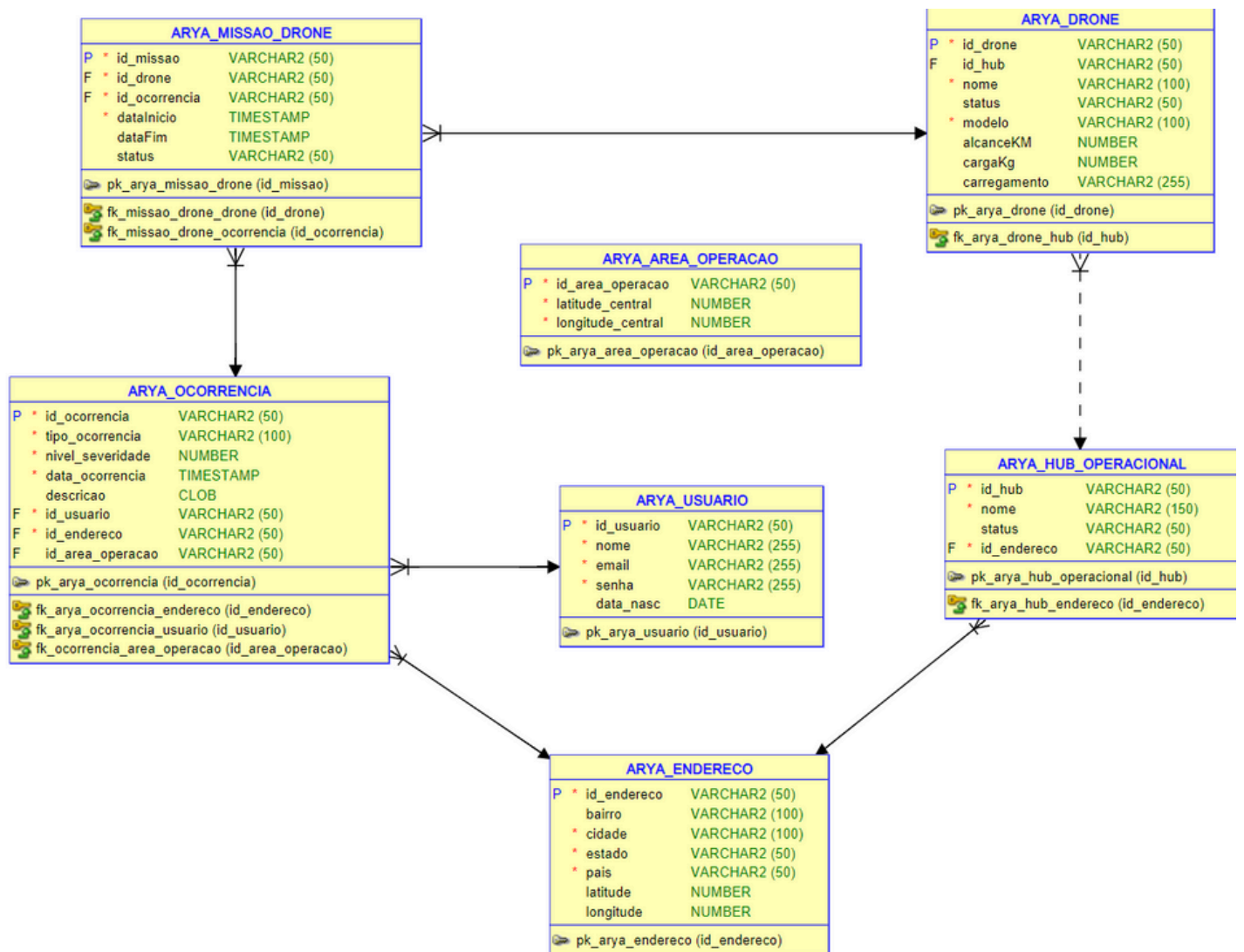
```
END IF;  
  
IF p_status IS NOT NULL AND LOWER(p_status) NOT IN ('concluída', 'em andamento', 'cancelada') THEN RAISE_APPLICATION_ERROR(-20021, 'Status inválido para Missão. Use: concluída, em andamento, cancelada.');
```

```
END IF;  
  
END prc_valida_missao_drone;
```

```
END pkg_arya_management;
```

Package Body PKG_ARYA_MANAGEMENT compilado

Modelagem :



MongoDB :

Global_Solution				
Banco_Testes > Global_Solution				
Sort by Collection Name				
View				
desastres_naturais				
Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
278.53 kB	2 K	530.00 B	1	69.63 kB
relatorios_desastres				
Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
45.06 kB	300	384.00 B	1	36.86 kB

Collection 1 : desastres_naturais.

desastres_naturais +

Banco_Teste > Global_Solution > desastres_naturais Open MongoDB shell

Documents 2.0K Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE 25 1 - 25 of 2000 ↺ ↻ ↷ ≡ { } ⌂

```
{
  "_id": "6843776014301ada21d6c49b",
  "id_zona": "Z_0003_SUD",
  "latitude": -24.49836,
  "longitude": -49.77282,
  "data": "2024-04-15T00:00:00.000+00:00",
  "precipitacao_mm": 163.4,
  "temperatura_c": 19.8,
  "umidade_percentual": 86.3,
  "densidade_populacional": 2379.6,
  "altitude_metros": 184.7,
  "declividade_graus": 38.7,
  "distancia_agua_km": 8.1,
  "tipo_de_solo": "rochoso",
  "uso_do_solo": "urbano",
  "nivel_acessibilidade": "alta",
  "frequencia_sismos": 0,
  "tipo_evento": "deslizamento",
  "ocorrenca": 0,
  "nivel_de_risco": "BAIXO",
  "mes": 4,
  "estacao_do_ano": "outono",
  "regiao": "Sudeste_Montanhoso_Litoraneo"
}
```

Collection 2 : relatorios_desastres:

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

</>

Options

ADD DATA

EXPORT DATA

UPDATE

DELETE

25

1 - 25 of 300

↺

↻

↷

⌵

☰

{ }

|

⌵

```
_id: ObjectId('6843758d14301ada21d6c36d')
idRelatorio: 2
dataHora: "2025-06-01 17:16:45"
latitude: 1.278283
longitude: -68.0108
tipoFonte: "Equipe de Campo"
tipoObservacao: "Deslizamento"
nivelGravidade: "Alta"
qtdAtingidos: 133
danoInfraestrutura: "Leve"
acessibilidade: "Fácil"
notasAdicionais: "Deslizamento atingiu parte de uma residência."
classificacaoRisco: "Suporte Necessário"
```

```
_id: ObjectId('6843758d14301ada21d6c36e')
idRelatorio: 3
dataHora: "2025-05-10 00:04:58"
latitude: -28.374458
longitude: -49.539387
tipoFonte: "App Usuário"
tipoObservacao: "Deslizamento"
nivelGravidade: "Baixa"
qtdAtingidos: 56
danoInfraestrutura: "Severo"
acessibilidade: "Fácil"
notasAdicionais: "Encosta cedeu parcialmente, risco de novos deslizamentos."
classificacaoRisco: "Suporte Necessário"
```